

Agent Harness

企业决策者精简版

研究修订稿

资料截面

2026-08-28

执行摘要

执行摘要	2
一、为什么 Harness 比模型更难	3
二、市场产品给出的五种答案	4
三、目标架构	5
四、安全原则	6
五、完成与证据	7
六、多 Agent 的决策门	8
七、Agent 如何进化	9
八、五级成熟度	10
九、建议路线	11
十、管理层应追踪的指标	12
十一、未来十二个月优先事项	13
结论	14

企业部署 Agent 的主要瓶颈正在从“模型能不能回答”转向“系统能不能安全、稳定地完成真实任务”。决定结果的不只是模型，而是模型、Harness、环境和反馈的乘积。Harness 负责把人的意图转成任务，把企业状态转成上下文，把模型动作转成受控副作用，再用外部证据判定完成。

建议企业采取“购买优秀 runtime，自建供应商无关控制面，保留逐层替换权”的战略。短期接入 Claude Code、Codex 等成熟 Agent；中期统一身份、策略、沙箱、任务合同、证据和评测；长期再根据真实数据选择性自研上下文、工具、loop 与进化系统。

一、为什么 Harness 比模型更难

模型调用是无状态概率计算，企业任务却需要持久状态、权限、恢复、外部副作用和责任。模型说“完成”不代表代码通过测试、报告数字可复核或外部 API 已唯一生效。生产系统必须区分：模型停止、候选答案、验证成功和副作用提交。

TEXT

Agent System Capability = Model × Harness × Environment × Feedback

任一项接近零，整体失效。更强模型无法弥补错误权限、坏环境或错误 verifier；复杂 Harness 也无法无限跨越模型能力边界。

二、市场产品给出的五种答案

产品	最值得借鉴的设计	企业需外置的部分
Claude Code	薄 loop、项目上下文、skills/hooks/MCP、沙箱	租户策略、业务完成门、统一证据
OpenAI Codex	App Server、多客户端核心、worktree、exec policy	业务身份、领域验证、供应商退出
Cursor	IDE 原生上下文、模型特定适配、云 VM	数据治理、跨产品控制面
DeepSeek Harness	Cordis 插件树、生命周期所有权、分层配置	独立 evaluator、发布治理、生产稳定性
OpenHands	Agent/Runtime 分离、事件与开放执行面	企业 IAM、策略、SLO 与运营

市场正在共同收敛到持久会话、动态上下文、结构化工具、隔离执行、策略审批、多 Agent、trace 与 eval。竞争重点不再是“有没有工具调用”，而是组合质量。

三、目标架构

TEXT

体验层	IDE / Web / CLI / API
控制面	任务合同 / 身份 / 策略 / 调度 / 审批
Agent 层	Claude/Codex/OpenHands/DSH/自研 adapter
执行面	workspace / sandbox / tool gateway / credential broker
证据面	artifact / trace / verifier / effect ledger
进化面	eval / mutation / canary / rollback / lineage

平台的稳定资产是 canonical contracts：Task、Action、Observation、Artifact、PolicyDecision、Checkpoint、VerificationResult 和 EvidencePackage。供应商 adapter 负责协议映射，不让厂商消息格式成为企业领域模型。

四、安全原则

Prompt 不是安全边界。“不要访问生产”只是概率建议；独立身份、最小权限、文件与网络沙箱才是确定性控制。凭证不进入模型上下文，而由 broker 在批准动作执行时注入短期、窄范围 token。

批准应针对具体 capability，包括目标资源、动作、期限和理由。多 Agent 委派时权限只能衰减，子 Agent 默认不能继续委派。MCP、skill 和 plugin 作为供应链依赖，需要版本固定、权限 manifest、扫描、签名、测试与撤销。

五、完成与证据

每个任务在入口形成 Completion Contract：目标、交付物、不变量、验收检查、证据、预算、freshness 和停止策略。Runtime 只能提交候选，可信 verifier 在隔离环境重算，commit controller 决定是否让副作用生效。

最终交付不是一段回答，而是：

TEXT

交付物 + 输入快照 + 检查结果 + 策略决定
+ 外部效果 + 未决风险 + 可重放引用

公开 benchmark 只能做能力探针。SWE-bench Verified 从人工修订到后来因测试缺陷与污染退役，说明 evaluator 本身也必须版本化和治理。

六、多 Agent 的决策门

多 Agent 不是默认升级。它适合高价值、可并行、上下文可隔离、需要独立验证的任务；不适合高度串行、频繁共享写状态和低价值任务。Anthropic 报告其内部 Research 系统的多 Agent 运行约使用普通聊天 15 倍的 token；这是厂商特定任务和实现下的数据，不是普遍定律，但足以说明选型必须与同成本的强单 Agent 和多 trial 基线比较。

任务应按可验证 artifact 拆分，不按“架构师/开发者/经理”等拟人职位拆分。每个委派包含目标、输入、输出 schema、工具、权限、预算和验收；并行工作区独立，合并后重新验证。

七、Agent 如何进化

进化分四层：任务内搜索与修复；跨任务 memory/skill；Harness 的 prompt、工具、上下文和流程；模型参数训练。越往后影响越大、反馈越慢。

可信闭环是：轨迹采集—失败归因—最小候选—隔离多 trial 评测—统计与安全门—canary—晋级或回滚。候选系统不能修改自己的 evaluator、held-out 数据、权限根和发布控制器。

短期最现实的“自我进化”是自动提出候选和自动评测，由策略或人批准发布，而不是生产 Agent 任意改写自己。

八、五级成熟度

等级	判定标准
L0 对话增强	单轮或简单工具，无外部完成证据
L1 受控执行	workspace、基础权限与日志
L2 可验证任务	completion contract、verifier、evidence
L3 平台化运行时	多 runtime、durable、租户级控制面
L4 受控进化	独立 eval、canary、rollback、lineage

成熟度按最弱关键层判断。拥有多个 Agent 或强模型不能让平台跳级。

九、建议路线

第一阶段用 adapter 统一接入现有 Agent，禁止业务散接。第二阶段外置完成门和证据。第三阶段统一 sandbox、tool gateway 与 credential broker。第四阶段从真实任务建立 capability、regression 和 safety eval。第五阶段逐层替换最有差异化价值的上下文、工具或 loop。第六阶段才引入自动 Harness 候选与受控进化。

十、管理层应追踪的指标

不要只看 token、调用量或“接受率”。核心指标包括：经外部验证的任务完成率；连续成功 pass^k ；错误完成率；人工总处理时间；每次验证成功成本；关键路径时延；权限升级与拒绝；恢复成功率；关键任务切片退化；canary 回滚和 verifier 完整性事件。

十一、未来十二个月优先事项

- 选定三类真实任务：仓库工程、企业分析和一个高价值工作流。
- 建立 canonical task/event/artifact/evidence schema。
- 接入两个不同 runtime，避免单厂商架构绑定。
- 建立身份、短期凭证、文件与网络沙箱。
- 把“模型说完成”替换为独立完成门。
- 从生产失败构建回归与安全 eval。
- 对多 Agent 以同成本 baseline 做收益证明。
- 只在上述基础稳定后开展 DSH 类可演化 Harness 实验。

结论

最好的企业 Harness 不是功能最多，也不是自治时间最长，而是在特定任务分布上，以最小上下文、最小权限、可接受成本和低协调熵，持续产生可验证结果。企业应拥有任务定义、权力边界、证据和学习数据；模型与 Agent runtime 可以采购，也应当可以替换。